



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Dianna Francesca M. Pardilla
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 2, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```

}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↴

```
import random
import agentpy as ap
import matplotlib.animation as animation
import matplotlib.pyplot as plt

APP_TITLE = "GridDrift: Multi-Agent Random Walk Simulation"

# < CONSOLE BANNER & HEADER >
print(f" {APP_TITLE} ")
print(" CS0065 (Intelligent Systems) | Section: AN41")
print(" Programmed by: Dianna Francesca M. Pardilla")
print(" Date: October 2, 2026")
print("~" * 60 + "\n")

# 1. < USER INPUTS >
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# 2. < AGENT DEFINITION (LAB TASKS) >
class CustomWalker(ap.Agent):
```

```

def setup(self):
    self.trajectory = []

def step(self):
    # Rule change: Weighted drift (40% right, 20% left, up, down)
    moves = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    weights = [0.40, 0.20, 0.20, 0.20]
    direction = random.choices(moves, weights=weights, k=1)[0]

    x, y = self.position

    # Keep agents inside grid limits
    bounded_x = max(
        0, min(self.model.p.grid_size[0] - 1, x + direction[0])
    ) # [cite: 1]
    bounded_y = max(
        0, min(self.model.p.grid_size[1] - 1, y + direction[1])
    ) # [cite: 1]

    self.position = (bounded_x, bounded_y)
    self.trajectory.append(self.position)

# 3. < MODEL DEFINITION >
class RandomWalkModel(ap.Model):

    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, CustomWalker)

        for agent in self.agents:
            init_pos = (
                random.randint(0, self.p.grid_size[0] - 1),
                random.randint(0, self.p.grid_size[1] - 1),
            )
            agent.position = init_pos
            agent.trajectory.append(init_pos)

        self.grid = ap.Grid(self, self.p.grid_size, torus=False)

```

```

self.grid.add_agents(self.agents)

def step(self):
    self.agents.step()

def end(self):
    print("\n" + "~" * 60)
    print("
                                FINAL POSITIONS SUMMARY
                                ")
    print(" Programmed by: Dianna Francesca M. Pardilla | AN41
                                ")
    print("~" * 60 + "\n")
    for i, agent in enumerate(self.agents, 1):
        start_coord = agent.trajectory[0]
        final_coord = agent.position
        print(
            f"Agent {i:02d} | Start: {start_coord} | Final: {final_coord} |
Total"
            f" Steps: {len(agent.trajectory) - 1}"
        )
    print("=" * 60 + "\n")

# 4. < EXECUTION & ANIMATION >
parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps,
}

model = RandomWalkModel(parameters)
model.setup()

fig, ax = plt.subplots(figsize=(7, 7))

def update(frame):
    if frame > 0:
        model.step()

```

```

ax.clear()
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True, linestyle="--", alpha=0.5)

# Clean title header on the plot window
ax.set_title(
    f"{APP_TITLE}\n"
    "Programmed by: Dianna Francesca M. Pardilla | CS0065 - AN41\n"
    f"Step: {frame}/{model.p.steps} | Date: October 2, 2026",
    fontsize=10,
    pad=10,
)

# Draw movement trails
for agent in model.agents:
    xs, ys = zip(*agent.trajectory)
    ax.plot(xs, ys, alpha=0.45, linewidth=1.5)

# Draw current positions
cx = [agent.position[0] for agent in model.agents]
cy = [agent.position[1] for agent in model.agents]
scat = ax.scatter(
    cx, cy, s=120, c="crimson", edgecolors="black", zorder=3,
    label="Agents"
)

if frame == model.p.steps:
    model.end()

return (scat,)

ani = animation.FuncAnimation(
    fig, update, frames=model.p.steps + 1, repeat=False, interval=200

```

```
)  
  
plt.tight_layout()  
plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?



Answer the Questions

1. Since the code doesn't stop the dots from walking into each other, a bunch of them just lands on the exact same tile at the same time, making them stack up and look like a big cluster. Wherein the grid gets super cramped and crowded to look at.
2. At first, they just stick close to where they spawned, but the longer the simulation runs, the more they spread out across the whole board instead of staying in one spot. When you give them more steps, they actually have time to wander around.
3. No, they can't escape at all. The code uses `max()` and `min()` to trap them inside. So whenever a dot tries to walk past the borders (like past 0 or the edge of the grid), the code just pushes it right back against the wall.
4. Since the dots bounce around and slowly fill up the space without leaving the borders, it works just like real-world situations where things drift or wander around an enclosed area, such as:
 - a. **Bumper cars or arcade pinballs:** It works like toy cars or arcade balls bouncing around an enclosed arena—they keep moving, collide or change direction, and bounce back whenever they hit the surrounding borders.
 - b. **People leaving a concert or mall:** It models how people walk out into an open area or parking lot when there are no designated lines to follow, wandering around in different directions until they disperse.
 - c. **Crowds in a hallway:** It mimics how people bump around and walk through a packed hallway or school cafeteria when there isn't a single line to follow.
 - d. **Fish swimming in a tank:** It mimics how a school of fish or bugs scatter across an enclosed space without a set leader or strict path.